Computational modelling of terrains

Hugo Ledoux
Ken Arroyo Ohori
Ravi Peters
Maarten Pronk

v2026.0

© 2026 Hugo Ledoux, Ken Arroyo Ohori, Ravi Peters, and Maarten Pronk

This work is available under a Creative Commons Attribution 4.0 International Licence. For licence details, see http://creativecommons.org/licenses/by/4.0/

v2026.0 [2026-10-30]
v2025.0 [2025-10-27]
v2024.0 [2024-10-29]
v2023.0 [2023-11-12]
v0.9 [2022-11-14]
v0.8 [2021-11-08]
v0.7 [2020-11-09]
v0.6 [2019-11-11]

Download latest version. The latest version of this book can be downloaded in PDF at https://github.com/tudelft3d/terrainbook/releases

Open book. The source code of the book, in Typst, is available at https://github.com/tudelft3d/terrainbook

Errors? Feedback?. Please report errors or potential improvements at https://github.com/tudelft3d/terrainbook/issues

Visibility queries on terrains

10

10.1 General problem 116
10.2 Visibility in terrains 116
10.3 Line-of-sight 117
10.4 Viewshed 118
10.5 Sky-view factor 119
10.6 Notes and comments 122
10.7 Exercises 122

youtu.be/3HYlBogztOU

Several applications using terrains involve *visibility queries*, ie given a viewpoint, which area of the surrounding terrain is visible? Examples of such applications are many: optimal position of telecommunication towers, path planning for hiking (to ensure the nicest views), estimation of the view for scenic drives, estimation of visual damage when trees in a forest are cut, etc. There are also several related problems. Two closely related examples are the estimation of shadows (position of the sun continually varies, also with seasons) and the calculation of the solar irradiance (how much sunlight will a certain area have per day/week). These can be used to estimate the photovoltaic potential (where can we best install solar panels?), for estimating where snow will accumulate in mountains, or for estimating the temperature of the ground (necessary for climate modelling), among other applications.

When referring to visibility problems in terrains, we address the following three fundamental problems:

line-of-sight (LoS) given a viewpoint v and another point q, does v see q (and vice-versa)? Or, in other words, does the segment vq intersect the terrain? The result is either True or False. (Figure 10.1)

viewshed given a viewpoint v, which area of the surrounding terrain is visible? The result is a polygon (potentially disconnected) showing the locations and extent of what is visible from v. Usually the extent is limited to a certain "horizon", or radius of visibility. If the terrain is formed of different objects (eg buildings), an object is either visible or not (simple case), or parts of objects can be visible (more complex). (Figure 10.2)

sky-view factor (SVF) given a viewpoint v, the SVF is the portion of the sky that is visible from v, ie the part of the sky hemisphere that is not obstructed by the surrounding terrain (and the objects on it, such as buildings and trees). (Figure 10.3)

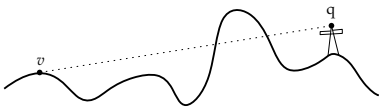


Figure 10.1: Line-of-sight between v and q; q is not visible.

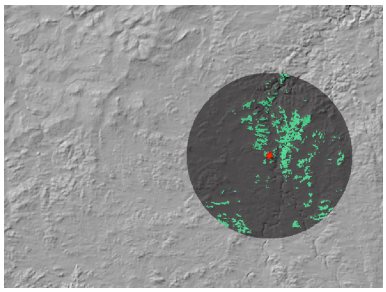


Figure 10.2: The viewshed at the location marked with a red star (green = visible; maximum view distance (dark grey) is set to 15 km).



Figure 10.3: A fisheye view looking straight up: every direction of the sky visible from the camera position appears in the image, which is essentially what a sky-view factor measures. Adapted from a photo by Autopilot, remapped by Peter Wieden (CC BY-SA 3.0), via Wikimedia Commons.

Observe that for all these problems, the viewpoint can either be directly on the terrain (at relative elevation 0 m) or at a given height (2 m for a human, or 30 m for a telecommunication tower).

We discuss in this chapter the general problem of visibility as defined in computer graphics, and then discuss how terrains, being 2.5D surfaces, simplify the problem.

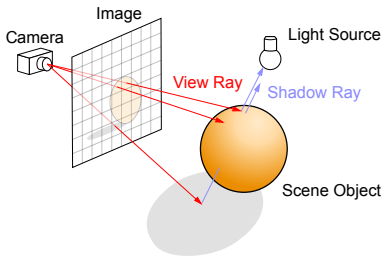


Figure 10.4: Ray tracing builds the image pixel-by-pixel by extending rays into the scene. (Figure from Wikipedia)

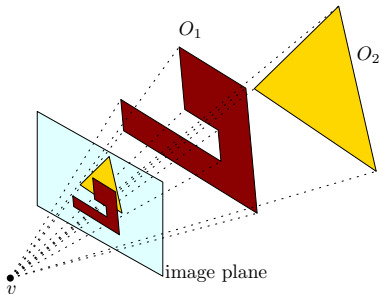


Figure 10.5: Two planar objects O_1 and O_2 are partially overlapping when viewed from v.

ray casting

hidden-surface determination

BSP-tree: binary space partitioning

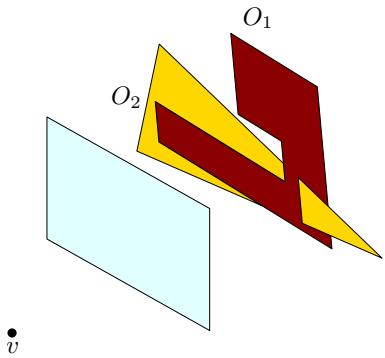


Figure 10.6: Part of O_2 is behind O_1 and part is in front.

10.1 The general visibility problem

Rendering is the process of generating (2D) images from 2D or 3D scenes. As shown in Figure 10.4, it involves projecting the (3D) objects in a scene to an image (say 800 × 800 pixels) and assigning one colour to each pixel. In the simplest case the colour assigned is that of the closest object, but to obtain photorealistic images, lighting, shading, and other physics-based functions are often applied (however this goes beyond the scope of this book).

Ray casting is used for each pixel: a ray is defined between the viewpoint v and the centre of the pixel, and the closest object in the scene must be found. The main issue involves finding that closest object, and especially discarding the other objects lying behind it (an operation usually called hidden-surface determination/removal). Figure 10.5 shows the idea for 2 objects (O_1 and O_2). Observe that objects can *partially* be hidden by others, and that the value of the pixel should always contain the closest object at that location.

One often used algorithm for solving the hidden-surface determination is the *depth-sort method*. Its main idea is to define a coordinate reference system with x and y on the viewing plane, and z perpendicular to it. The objects are first sorted according to their maximal z-values, and the objects are drawn on the viewing plane from the furthest to the closest. The value of a given pixel could therefore be redrawn several times, but its value will contain the colour of the closest object. The algorithm assumes that all objects are planar polygons, which is for the case of terrains not an issue. Observe that this algorithm is often referred to as the *painter's algorithm* since it mimics the way one would draw a scene: details in the foreground are drawn "over" the background.

It should be noticed that it is possible that objects cannot be strictly z-ordered since their z-values can overlap. Figure 10.6 shows one example: the object O_2 from Figure 10.5 was slightly rotated and now part of it is in front of O_1 and part of it is behind. The solution to this is to decompose one of the objects by the plane of the other, and to process all the parts as different objects.

An efficient implementation of the depth-sort algorithm requires indexing the objects in the scene, and for this BSP-trees are commonly used. A depth order for the scene can now be obtained by a traversal of the BSP tree.

10.2 Visibility in terrains

For 2.5D terrains, the visibility problem is simplified because:

1. cells are simple polygons, usually triangles or squares;
2. we can sort those cells from closest to farthest;
3. we can convert the problem to a 2D one (because a terrain is a 2.5D surface).

We can then exploit the connectivity and adjacency between the 2D cells forming a terrain to minimise the number of objects to test (for intersections and for hidden-surface determination).

10.2.1 Visibility in grids

Solving visibility queries in grids is simpler than with triangles since the topology of the grid is implied (we have direct access to the neighbours of a given cell), and because grid cells are usually small we can assume that a grid cell is visible (or not) if its centre is visible (or not). The same assumption is tricky for triangles, since these can be large; in practice it is often assumed that a triangle is visible if its 3 vertices are visible, but this varies per implementation.

We describe below how both LoS, viewshed, and SVF queries can be implemented for grids; the same principles could be applied to TINs with minor modifications.

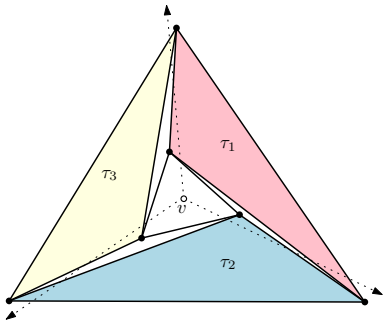


Figure 10.7: The 3 triangles τ_1, τ_2, and τ_3 form a cycle when viewed from the viewpoint v, and it is not possible to sort them from furthest to closest (without decomposing them).

10.2.2 Visibility in TINs

Using the depth-sort algorithm for arbitrary triangles would require using a BSP-tree for indexing and sorting the triangles, and some triangles would need to be decomposed, as explained above. Figure 10.7 shows one simple example.

However, it has been proven that Delaunay triangulations are *acyclic* for any fixed viewpoint. In other words, the in-front/behind relationship for the triangles of a DT, with respect to a given viewpoint, is acyclic (see Figure 10.8). Therefore, to obtain the triangles intersecting a ray coming out of a viewpoint (ordered from the closest to farthest), it suffices to modify slightly the point location algorithm from Section 4.4.2. This operation can be performed in 2D, by projecting the triangles of the TIN to the xy -plane.

This means that visibility queries in TINs—like in grids—are greatly simplified compared to the general case where the ordering of objects is the main difficulty (and handling overlapping objects like in Figure 10.6).

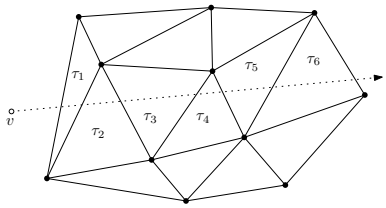


Figure 10.8: The triangles in a DT can be ordered in an in-front/behind manner when viewed from a viewpoint.

10.3 Line-of-sight

A LoS query, between a viewpoint v and another point q, implies reconstructing the profile of the terrain along the vertical projection of vq (let us call it vq_{xy}). It then suffices to follow vq and verify whether the elevation at any (x, y) location along the profile is higher than that of vq. As shown in Figure 10.9, since the terrain is discretised into grid cells, there are 2 options to reconstruct the profile between v and q:

1. identify all the cells intersected by vq_{xy}, and assign the centre of each cell by projecting it to the terrain profile. This is what is done in Figure 10.9.
2. consider the edges of the cells, collect all the edges that are intersected by vq_{xy}, and linearly interpolate the elevations. This is far more expensive to compute, and therefore less used in practice.

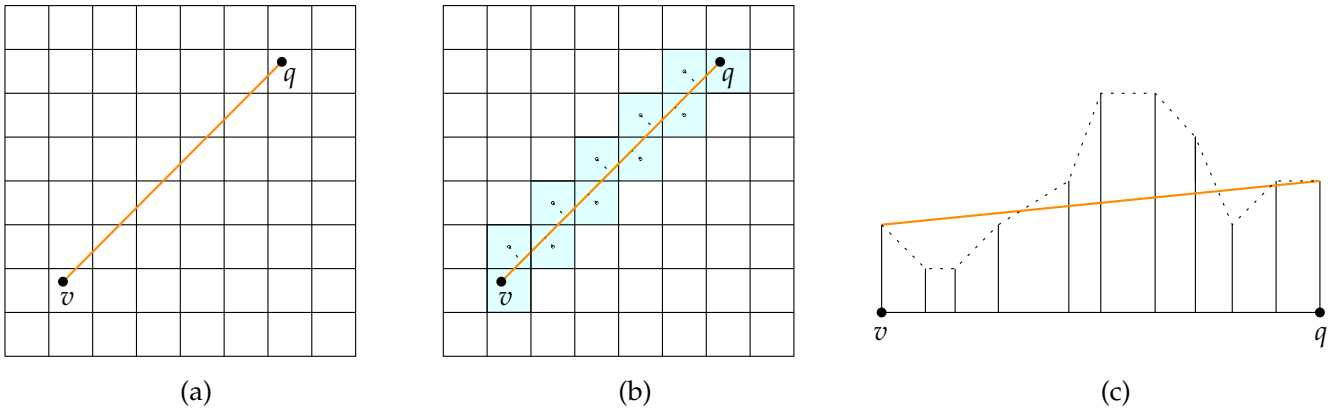


Figure 10.9: Line-of-sight between v and q. Observe that along the profile, the points with elevation are not equally spaced.

The algorithm is thus as follows. Start at v, and for each pixel c encountered along vq_{xy}, verify whether the elevation value of vq at that location is higher than the elevation of c. If it is, then continue to the next pixel; if not, then there is an intersection and thus the visibility is False. If the pixel containing q is reached without detecting an intersection, then the visibility is True.

10.4 Viewshed

As shown in Figure 10.10, computing the viewshed from a single viewpoint v implies that the LoS between v and the centre of each pixel in a given radius is tested. The result of a viewshed is a binary grid; in Figure 10.2, True/visible pixels are green, and False/invisible ones are dark grey.

While this brute-force approach will work, several redundant computations will be made, since several of the rays from v will intersect the same grid cells. Furthermore, depending on the resolution, the number of cells in a 5 km radius (a reasonable value where humans can see) can become *very* large. As an example, with the AHN gridded version (50 cm resolution), this means about 314 million queries ($\pi \times \left(\frac{5000}{0.5}\right)^2$).

One alternative solution is shown in Figure 10.10b: it involves sampling the grid cells intersecting the border of the visible circle (obtaining several

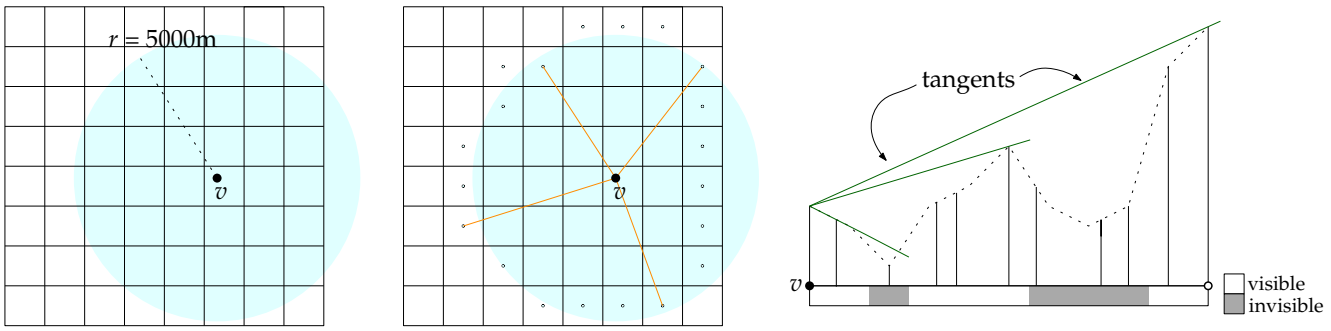


Figure 10.10: Viewshed for the point v; the blue circle is the radius of the horizon (5000 m in this case).

centres q_i), and computing the visibility of each of the cells along the line segment vq_i as we 'walk' from v. Observe that, along vq_i, it is possible that a point far from v is visible, while several closer points are not; Figure 10.10c gives an example.

One solution involves using so-called *tangents*. The current tangent t_{cur} is first initialised as a vector pointing downwards. Then, starting at v, we walk along the ray vq_i, and for each cell intersected its elevation z is compared to the elevation of t_{cur} at that location. If z is lower, then the cell is invisible. If z is higher, then the cell is visible and t_{cur} is updated with a new tangent using the current elevation.

Viewsheds with several viewpoints v_i are also very useful, think for instance of obtaining the viewshed along a road. This can be computed by sampling the road at every 50 m and computing the viewsheds from each of the points. Each viewshed yields a binary grid, and it suffices to use a map algebra operator to combine the results into one grid (if one cell is visible from any viewpoint, then it is visible).

10.5 Sky-view factor

The sky-view factor (SVF) is the fraction of the sky that is visible from a given location v on the ground (usually at the surface itself, or at pedestrian height, 2 m). It depends on the height and locations of the obstacles (eg buildings, trees, etc.) in the surroundings of v. The result is a normalised value between 0 (the sky is completely hidden) and 1 (the entire sky hemisphere is visible).

SVF is value in range [0,1]

The SVF is not only a geometric indicator: under the assumption that the sky brightness is approximately uniform, it is a proxy for the amount of diffuse sky radiation that a location receives, which explains its use in many applications. It is for instance used in urban climatology to quantify how open a street or a courtyard is (it correlates with the urban heat island effect in cities), to estimate the formation of frost on roads, or to estimate the availability of GPS signals in urban areas. It is also a useful terrain visualisation technique, as we explain in Chapter 9, since the SVF does not depend on any light direction, it does not suffer from the relief inversion that affects hillshading (see Section 9.4).

The SVF can be computed in different ways: from fisheye images, from a digital terrain model, or from point clouds (where the full 3D geometry is considered, since the 2.5D assumption no longer holds).

10.5.1 Fisheye images

As shown in Figure 10.11, a fisheye image captures the entire sky hemisphere in a single image: the centre of the image corresponds to the zenith, and its border to the horizon. To compute the SVF, the pixels of the image are first classified as sky or as obstruction. The image cannot however simply be counted as-is: because the hemisphere is projected onto the plane of the image, the pixels close to the border (near the horizon) represent a larger part of the sky than those close to the centre. The image is therefore divided into concentric rings and sectors (Figure 10.11c),



(a) (b) (c)

Figure 10.11: **(a)** Fisheye image showing the entire sky hemisphere (same image as Figure 10.3); **(b)** the pixels are classified as sky or as obstruction, and the SVF is computed from the sky pixels; in this case the SVF value is about 0.6, ie 60% of the sky hemisphere is unobstructed; **(c)** the image is divided into concentric rings and sectors; the cells towards the border (close to the horizon) are larger and represent a larger solid angle, and each sky cell is weighted by the solid angle that it represents.

each sky pixel is weighted by the solid angle of its cell, and the SVF is the ratio of the weighted sky to the weighted whole. This method is common in urban climatology and in forestry, but it captures the sky at only one location, while a terrain model (or a point cloud) allows computing the SVF everywhere at once.

10.5.2 Gridded terrains

The SVF can also be computed from a gridded terrain, using the same principles as the viewshed. For each of the n directions (azimuths) equally spaced around v, we 'walk' along the ray starting at v up to a maximum distance R (the search radius), and we store the vertical elevation angle γ_i of the horizon, ie the maximal angle under which the terrain is seen along that direction. Observe that this is exactly the tangent algorithm described above, except that instead of comparing each cell to the current tangent, we simply keep the largest angle encountered. The sky visible from v is the portion of the hemisphere lying above the horizon. Contrary to the fisheye method, no projection (and no tessellation of the sky) is needed: because a terrain is a 2.5D surface, a direction of the sky is visible from v if and only if its elevation angle is larger than the horizon elevation angle γ_i of its azimuth. The contribution of each sector can therefore be computed analytically, and only n angles need to be stored.

solid angle

Let us first consider the case where the horizon has the same elevation angle γ in every direction. The visible sky is then the part of the hemisphere above a cone with apex v, and its solid angle is $2\pi(1 - \sin\gamma)$ (the solid angle of the complete hemisphere is 2π). The visible portion of the sky is thus $(1 - \sin\gamma)$, and with n directions the sky-view factor is obtained by averaging this quantity:

$$\text{SVF} = 1 - \frac{1}{n}\sum_{i=1}^{n} \sin\gamma_i \tag{10.1}$$

The values range from 1 (the entire hemisphere is visible; this is the case on exposed locations such as peaks) to 0 (the sky is completely obstructed; this happens in deep sinks and at the bottom of deep valleys). In practice, the two parameters to set are the number of directions n (8 or 16 is common) and the search radius R, which should be chosen according to the

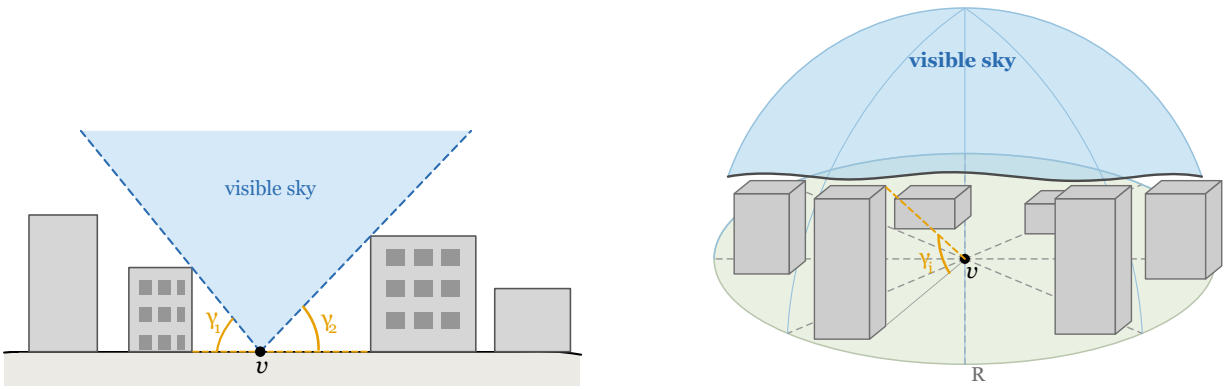


Figure 10.12: Sky-view factor computation. **(a)** Profile of a scene with obstacles on both sides of the viewpoint v: in a given direction, the horizon elevation angle γ_i is the angle above the horizontal (dashed) at which the line of sight is tangent to the horizon (here the rooftops). **(b)** The same scene in 3D with the $n = 8$ azimuths along which the horizon elevation angle γ_i is computed up to the search radius R.

scale of the features of interest. Observe that both parameters influence the result: with too few directions, narrow obstacles lying between 2 rays are missed, and with a search radius that is too small, the distant terrain is ignored; in both cases the terrain looks less obstructed than it is, and the computed SVF is thus an overestimation of the true value.

10.5.3 Using point clouds

Computing the SVF from a point cloud requires considering the full 3D geometry of the surroundings, including buildings, trees, and terrain. This is needed because with real 3D objects 1 horizon angle per direction no longer suffices: gaps in the canopy, and overhangs (eg balconies or bridges), mean that the sky visible in one direction cannot be summarised by a single angle. As for the fisheye method, the sky hemisphere is divided into cells, each weighted by its solid angle, and many rays are cast from the viewpoint to determine whether they intersect any of the points in the cloud. While this method can capture complex occlusion patterns more accurately than gridded approaches, it is computationally intensive, and the result depends on the density of the point cloud and on how the points are classified (vegetation points, for instance, are often given a transmissivity). In practice, a gridded DSM derived from the same point cloud is often used instead, as a compromise between accuracy and computation time.

⚙ Sky-view factor computation in practice

The QGIS plugin UMEP (Urban Multi-scale Environmental Predictor: https://umep-docs.readthedocs.io) provides tools for calculating the sky-view factor from gridded DSMs.

The tool allows for separate analysis of building and tree contributions, assuming a transmissivity of light through the vegetation based on tree species.

10.6 Notes and comments

The ‘tangent algorithm’ to compute viewsheds was first described by Blelloch (1990).

The description here is inspired by that of De Floriani & Magillo (1999).

Newell et al. (1972) first proposed the depth-sorting algorithm and the decomposition necessary when polygons in the scene cannot be sorted from furthest to closest.

Edelsbrunner (1990) proved that Delaunay triangulations, in any dimensions, are acyclic.

A good explanation of the fisheye projection (from hemisphere to a plane) can be found at https://paulbourke.net/dome/fisheye/.

The QGIS plugin UMEP is based on the paper Lindberg et al. (2018).

10.7 Exercises

1. Explain why the spacing in Figure 10.9c along the profile has points that are not equally spaced.
2. The sky-view factor of a location is computed with $n = 8$ equally spaced directions. What is the SVF if the horizon elevation angle is 30° in all directions? And if it is 0° in 4 directions and 60° in the 4 others?
3. In a fisheye image, do the pixels close to the centre or those close to the border of the image represent a larger part of the sky? What would happen to the SVF if one simply counted the sky pixels without weighting them?
4. You are given a 2.75D terrain of an area, it is composed of triangles, and your aim is to perform line-of-sight queries between some locations. Describe the algorithm that you will implement to perform the queries.